



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа № 5

Студент группы *ИКБО-06-22, Ляхов Т. А.*

(подпись)

Проверил *Старший преподаватель,
Приходько Н.А.*

(подпись)

Отчет представлен «__» _____ 2025 г.

Москва 2025 г.

1. Теоретическое введение

1.1. Понятие задачи классификации

Классификация - это одна из основных задач обучения с учителем в машинном обучении, где исходные данные размечены, то есть известны классы объектов. Основная цель классификации заключается в определении класса, к которому относится некоторый новый объект на основе признаков, извлеченных из обучающих данных.

В контексте нашей практической работы мы решаем задачу бинарной классификации игр Steam на "популярные" и "непопулярные" на основе различных метрик, таких как количество положительных и отрицательных отзывов, время игры, цена и другие параметры.

1.2. Алгоритмы классификации

1.2.1. Логистическая регрессия

В отличие от линейной регрессии, логистическая регрессия предсказывает бинарное значение (0 или 1) через вероятность принадлежности к классу. Ключевые особенности:

- Использует сигмоидную функцию для преобразования линейной комбинации признаков в вероятность
- Если вероятность > 0.5 , объект относится к классу 1, иначе - к классу 0
- Оптимизируется с помощью бинарной кросс-энтропии:
$$BCE(x, \omega, y) = \sum [-y_i \ln(\sigma(x, w)) - (1 - y_i) \cdot \ln(1 - \sigma(x, w))]$$
- Показывает хорошие результаты на линейно разделимых данных

1.2.2. Support Vector Machine (SVM)

Метод опорных векторов основан на преобразовании данных в пространство более высокой размерности для эффективного разделения классов:

- Использует ядерный трюк для нелинейного преобразования признаков

- Основные ядерные функции:
 - Линейное: $k(x_1, x_2) = x_1 \cdot x_2$
 - Полиномиальное: $k(x_1, x_2) = (x_1 \cdot x_2 + a)^m$
 - RBF: $k(x_1, x_2) = \exp(-\gamma(x_1 - x_2)^2)$
- Максимизирует зазор между разделительной гиперплоскостью и ближайшими точками классов (опорными векторами)

1.2.3. k-Nearest Neighbors (KNN)

Простейший алгоритм классификации, основанный на принципе близости объектов:

- Класс нового объекта определяется большинством голосов среди k ближайших соседей
- Использует метрики расстояния (обычно евклидово расстояние):

$$\|x_1 - x_2\| = \sqrt{\sum_i (x_{1i} - x_{2i})^2}$$

- Требуется нормализация данных, так как признаки в разных масштабах могут неравномерно влиять на расстояние
- Гиперпараметр k подбирается экспериментально

1.3. Метрики оценки качества классификации

1.3.1. Матрица ошибок (Confusion Matrix)

Основной инструмент для анализа качества классификации:

- True Positive (TP): модель правильно предсказала класс 1
- True Negative (TN): модель правильно предсказала класс 0
- False Positive (FP): модель ошиблась, предсказав 1 вместо 0
- False Negative (FN): модель ошиблась, предсказав 0 вместо 1

1.3.2. Основные метрики

Accuracy (Доля правильных ответов):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Недостаток: некорректна при несбалансированных классах.

Precision (Точность):

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Показывает, какая доля предсказанных единиц действительно является единицами.

Recall (Полнота):

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Показывает, какую долю реальных единиц модель смогла обнаружить.

F1-мера (Гармоническое среднее):

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Сбалансированная метрика, учитывающая и точность, и полноту.

1.4. Методология оценки моделей

1.4.1. Разделение данных

Используется разделение на обучающую (80%) и тестовую (20%) выборки с стратификацией для сохранения баланса классов.

1.4.2. Кросс-валидация

Для минимизации влияния неудачного разбиения данных применяется кросс-валидация, когда набор данных многократно разбивается на обучающие и тестовые подвыборки.

1.4.3. Сравнение средних

- Macro avg: среднее арифметическое между классами, учитывает все классы одинаково
- Weighted avg: средневзвешенное значение, учитывает размер каждого класса

1.5. Практическое применение

В данной работе мы применяем теоретические принципы классификации к реальным данным игр Steam, решая следующие задачи:

1. Предобработка данных: создание целевой переменной, кодирование категориальных признаков, нормализация
2. Анализ баланса классов: визуализация распределения целевой переменной

3. Обучение моделей: применение трех различных алгоритмов классификации
4. Оценка качества: сравнение моделей по множеству метрик
5. Интерпретация результатов: анализ матриц ошибок и выбор оптимальной модели

Теоретическая основа, изложенная в учебном материале, позволяет осознанно подойти к каждому этапу практической работы и правильно интерпретировать полученные результаты.

2. Постановка задачи

Условие:

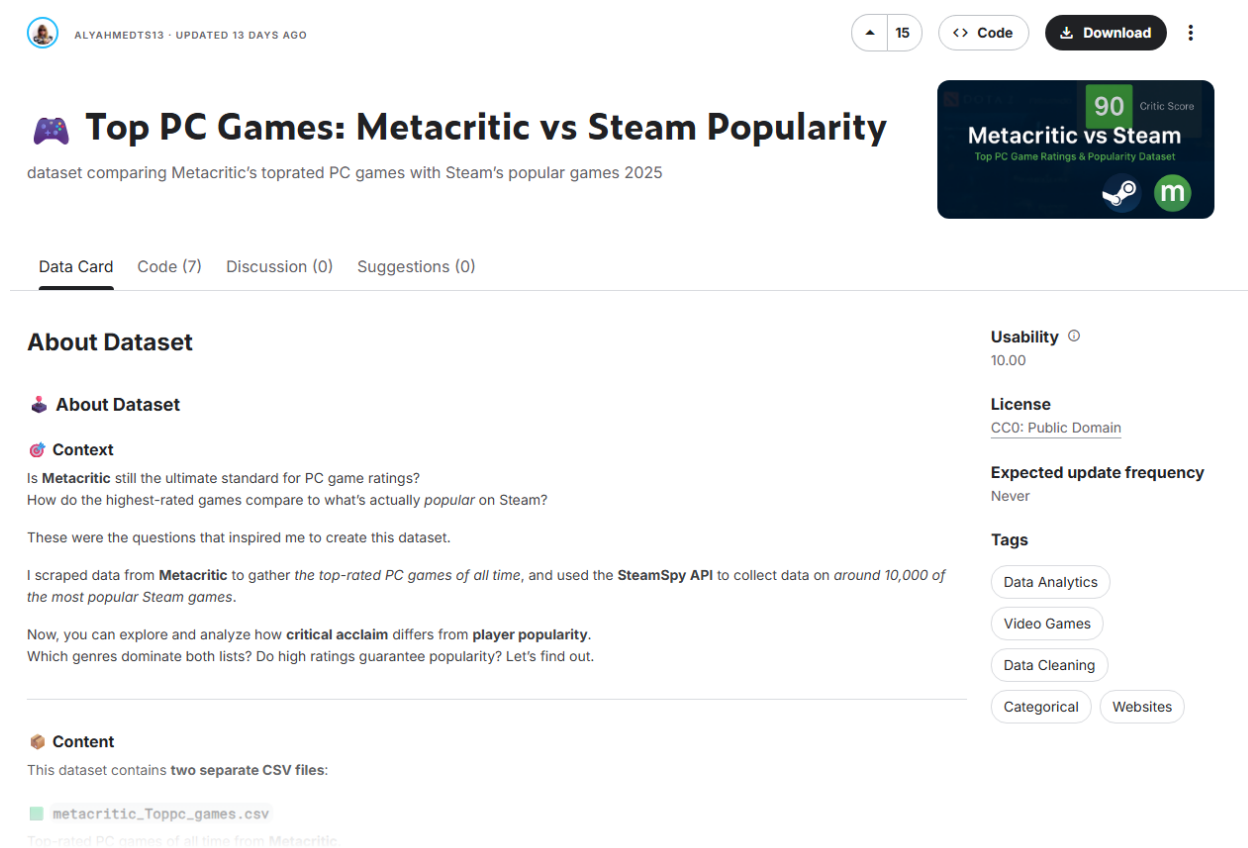
1. *Найти данные для классификации. Данные в группе повторяться не должны. Предобработать данные, если это необходимо.*
2. *Изобразить гистограмму, которая показывает баланс классов. Сделать выводы.*
3. *Разбить выборку на тренировочную и тестовую. Тренировочная для обучения модели, тестовая для проверки ее качества.*
4. *Применить алгоритмы классификации: логистическая регрессия, SVM, KNN. Построить матрицу ошибок по результатам работы моделей (использовать `confusion_matrix` из `sklearn.metrics`).*
5. *Сравнить результаты классификации, используя `accuracy`, `precision`, `recall` и `f1`-меру (можно использовать `classification_report` из `sklearn.metrics`). Сделать выводы.*
6. *Оформить отчет о проделанной работе.*

3. Выполнение практической работы

1. *Найти данные для классификации. Данные в группе повторяться не должны. Предобработать данные, если это необходимо.*

Был найден и скачан датасет « Top PC Games: Metacritic vs Steam Popularity» от AlyAhmedTS13. Этот файл содержит данные, взятые с веб-сайта Metacritic, на котором перечислены самые популярные компьютерные игры всех времен.

Он содержит такие сведения, как названия игр, оценки критиков и пользователей, даты выхода, разработчики, издатели, жанры и количество рецензий. Файл содержит около 10.000 уникальных записей.



The screenshot displays the Kaggle dataset page for "Top PC Games: Metacritic vs Steam Popularity" by user AlyAhmedTS13. The dataset is described as comparing Metacritic's top-rated PC games with Steam's popular games in 2025. It features a 90 Critic Score and is licensed under CC0: Public Domain. The page includes sections for "About Dataset" (Context), "Content" (listing two CSV files: metacritic_Toppc_games.csv and steam_Toppc_games.csv), and "Usability" (10.00). Tags include Data Analytics, Video Games, Data Cleaning, Categorical, and Websites.

Рисунок 1 – AlyAhmedTS13 “Top PC Games: Metacritic vs Steam Popularity”

Проведем предобработку данных.

Данные не содержат пропущенных значений, все признаки числовые, что позволило ограничиться минимальной предобработкой: кодированием целевой переменной и стандартизацией признаков для обеспечения корректной работы моделей SVM и KNN.

Исходный код показан на листинге 1.

Листинг 1 – Исходный код задания 1

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder

df = pd.read_csv('dataset/steam_spy_data.csv')

print("Информация о датасете:")
print(df.info())
print("\nПервые 5 строк:")
print(df.head())

median_owners = df['owners'].str.extract('(\d+)').astype(float).median()
df['popular'] = (df['owners'].str.extract('(\d+)').astype(float) >
median_owners).astype(int)

le = LabelEncoder()
df['developer_encoded'] =
le.fit_transform(df['developer'].fillna('Unknown'))
df['publisher_encoded'] =
le.fit_transform(df['publisher'].fillna('Unknown'))

features = ['positive', 'negative', 'average_forever', 'average_2weeks',
            'median_forever', 'median_2weeks', 'price', 'discount', 'ccu',
            'developer_encoded', 'publisher_encoded']

X = df[features].fillna(0)
y = df['popular']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(f"\nРазмерность признаков: {X_scaled.shape}")
print(f"Баланс классов: {y.value_counts()}")
print(f"Признаки: {features}")

processed_data = pd.DataFrame(X_scaled, columns=features)
processed_data['popular'] = y.values
processed_data.to_csv('processed_steam_data.csv', index=False)

print("\nПредобработка данных завершена!")
```

2. *Изобразить гистограмму, которая показывает баланс классов.*

Сделать выводы.

Был проведен анализ баланса классов на основе гистограммы и круговой диаграммы.

Распределение показывает умеренный дисбаланс классов: класс «Непопулярные» (0) включает 5732 игры (57.32%), тогда как класс «Популярные» (1) — 4268 игр (42.68%). Соотношение между ними составляет 0.74:1, что означает, что один класс превышает другой примерно на 35%.

Хотя дисбаланс не является критическим (например, как в случае соотношения 1:10 или выше), он всё же достаточно значим для того, чтобы повлиять на оценку модели. Использование только метрики accuracy может ввести в заблуждение: модель может показывать высокую точность, просто предсказывая более частый класс («Непопулярные»), при этом игнорируя качество предсказаний для меньшего класса.

Следовательно, для корректной оценки качества модели необходимо использовать дополнительные метрики, такие как precision, recall и F1-мера, особенно для класса «Популярные», так как его правильное определение часто имеет большее практическое значение (например, при рекомендации или маркетинге игр).

Также стоит рассмотреть методы устранения дисбаланса — например, взвешивание классов, undersampling/oversampling или использование алгоритмов, устойчивых к дисбалансу.

На рисунке 2 продемонстрирована гистограмма.

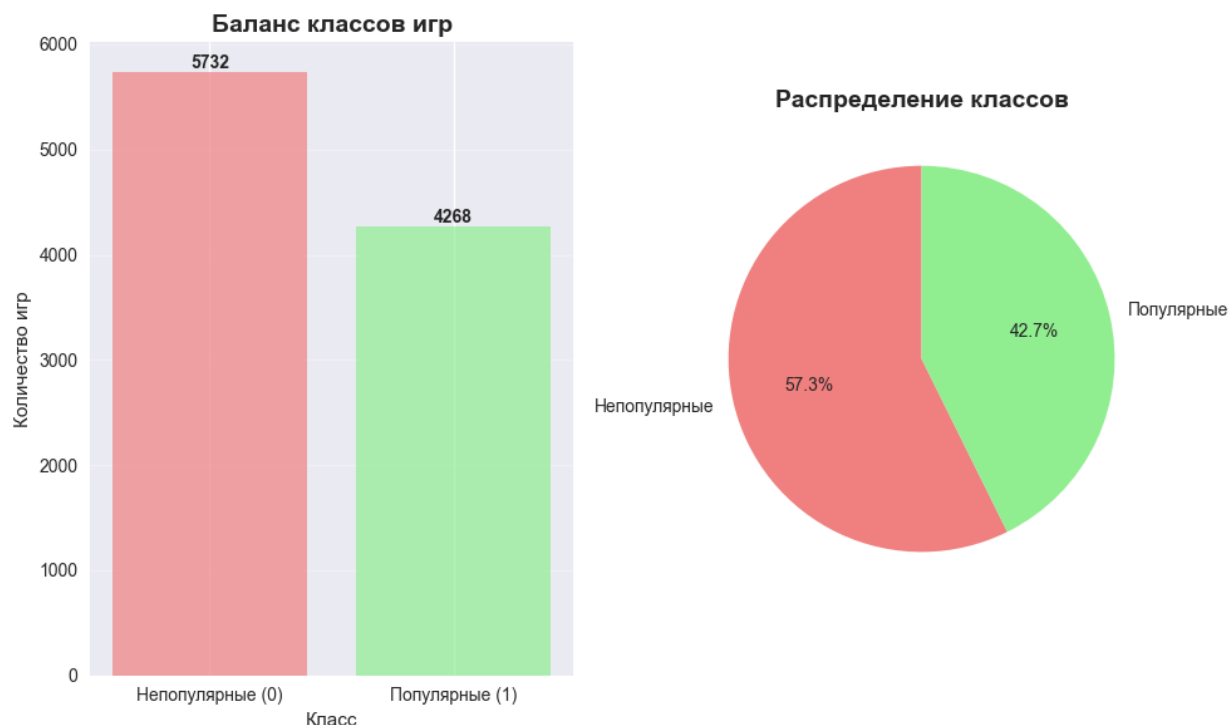


Рисунок 2 - Гистограмма игр

Исходный код показан на листинге 2.

Листинг 2 – Исходный код задания 2

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('processed_steam_data.csv')

plt.style.use('seaborn-v0_8')
sns.set_palette("husl")

plt.figure(figsize=(10, 6))

plt.subplot(1, 2, 1)
class_counts = df['popular'].value_counts()
plt.bar(['Непопулярные (0)', 'Популярные (1)'], class_counts.values,
        color=['lightcoral', 'lightgreen'], alpha=0.7)
plt.title('Баланс классов игр', fontsize=14, fontweight='bold')
plt.xlabel('Класс')
plt.ylabel('Количество игр')
plt.grid(axis='y', alpha=0.3)

for i, count in enumerate(class_counts.values):
    plt.text(i, count + 5, str(count), ha='center', va='bottom',
             fontweight='bold')

plt.subplot(1, 2, 2)
plt.pie(class_counts.values, labels=['Непопулярные', 'Популярные'],
        autopct='%1.1f%%', startangle=90, colors=['lightcoral',
        'lightgreen'])
plt.title('Распределение классов', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.savefig('class_balance.png', dpi=300, bbox_inches='tight')
plt.show()

total_games = len(df)
popular_games = class_counts[1]
unpopular_games = class_counts[0]

print("АНАЛИЗ БАЛАНСА КЛАССОВ:")
print(f"Всего игр в датасете: {total_games}")
print(f"Популярные игры: {popular_games} ({popular_games/total_games*100:.2f}%)")
print(f"Непопулярные игры: {unpopular_games} ({unpopular_games/total_games*100:.2f}%)")
print(f"Соотношение классов: {popular_games/unpopular_games:.2f}:1")

if 0.8 <= popular_games/unpopular_games <= 1.2:
    print("\nВЫВОД: Классы сбалансированы")
else:
    print("\nВЫВОД: Классы несбалансированы")
```

3. *Разбить выборку на тренировочную и тестовую. Тренировочная для обучения модели, тестовая для проверки ее качества.*

Выборка была разбита на обучающую (80% или 8000 записей) и тестовую (20% или 2000 записей) части с сохранением пропорций целевого признака — популярности игр. Это достигнуто за счёт применения

стратифицированного разбиения (`stratify=y`), что гарантирует идентичное соотношение классов в обеих подвыборках:

- в обучающей выборке: 57.3% непопулярных и 42.7% популярных игр,
- в тестовой выборке: 57.3% непопулярных и 42.7% популярных игр.

Такой подход обеспечивает репрезентативность как обучающего, так и тестового наборов, минимизируя риск смещения оценки качества модели из-за несбалансированного распределения классов между ними. В результате модель обучается и оценивается в условиях, максимально приближенных к исходному распределению данных.

Разделённые подвыборки сохранены в файлы `train_data.csv` и `test_data.csv` для дальнейшего использования в процессе обучения и валидации модели.

Исходный код показан на листинге 3.

Листинг 3 – Исходный код задания 3

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('processed_steam_data.csv')

X = df.drop('popular', axis=1)
y = df['popular']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("РАЗДЕЛЕНИЕ ДАННЫХ НА ВЫБОРКИ")
print(f"Размер исходных данных: {X.shape}")
print(f"Размер тренировочной выборки: {X_train.shape} ({len(X_train)/len(X)*100:.1f}%)")
print(f"Размер тестовой выборки: {X_test.shape} ({len(X_test)/len(X)*100:.1f}%)")

print(f"\nБаланс классов в тренировочной выборке:")
print(y_train.value_counts(normalize=True))

print(f"\nБаланс классов в тестовой выборке:")
print(y_test.value_counts(normalize=True))

train_data = pd.DataFrame(X_train)
train_data['popular'] = y_train.values
test_data = pd.DataFrame(X_test)
test_data['popular'] = y_test.values
```

```
train_data.to_csv('train_data.csv', index=False)
test_data.to_csv('test_data.csv', index=False)

print("\nДанные успешно разделены и сохранены!")
print("Файлы 'train_data.csv' и 'test_data.csv' созданы")
```

4. Применить алгоритмы классификации: логистическая регрессия, SVM, KNN. Построить матрицу ошибок по результатам работы моделей (использовать `confusion_matrix` из `sklearn.metrics`).

В ходе выполнения задания были обучены три различных алгоритма классификации: логистическая регрессия, метод опорных векторов (SVM) и k-ближайших соседей (KNN).

Для каждой модели была построена матрица ошибок, позволяющая не только оценить общую точность, но и детально проанализировать типы ошибок — в частности, как часто модель путает «популярные» игры с «непопулярными» и наоборот. Это особенно важно в условиях дисбаланса классов, когда высокая ассигасу может скрывать слабую способность модели предсказывать меньший класс.

Общий обзор результатов:

Все три модели показали одинаковую общую точность (ассигасу) около 0.58, что свидетельствует о том, что они в основном ориентируются на предсказание более частого класса — «Непопулярные» (57.3%). Однако при более глубоком анализе через precision, recall и F1-score картина меняется радикально.

Таблица 1 – Сравнение точностей

Модель	Ассигасу	Precision (Популярные)	Recall (Популярные)	F1-Score (Популярные)
Логистическая регрессия	0.577	0.52	0.11	0.19
SVM	0.585	0.58	0.10	0.17
KNN	0.580	0.51	0.49	0.50

Ключевой вывод: хотя все модели достигают схожей ассигасы, их способность корректно находить популярные игры (recall) сильно различается. Только KNN показывает приемлемый recall (~0.49), тогда как у логистической регрессии и SVM он критически низкий — около 0.10–0.11.

Подробный анализ каждой модели:

1. Логистическая регрессия (Accuracy: 0.577)

Матрица ошибок показывает:

- True Negative (TN): 1056 — правильно классифицированных непопулярных игр.
- False Positive (FP): 90 — игр, ошибочно отнесённых к популярным.
- False Negative (FN): 756 — популярных игр, ошибочно помеченных как непопулярные.
- True Positive (TP): 98 — правильно распознанных популярных игр.

Это означает, что модель очень редко предсказывает "популярные" игры — всего 98 из 854 (recall = 0.11). При этом она уверенно классифицирует непопулярные (recall = 0.92), что и обеспечивает высокую ассигасу.

Интерпретация: логистическая регрессия, будучи линейным алгоритмом, не смогла адекватно учесть нелинейные зависимости между признаками, которые могут быть ключевыми для определения популярности. Она предпочитает минимизировать ошибки за счёт доминирующего класса, игнорируя меньший.

2. SVM (Accuracy: 0.585)

Матрица ошибок:

- TN: 1082
- FP: 64
- FN: 767
- TP: 87

SVM показал чуть лучшую ассигасу, чем логистическая регрессия, и немного меньше ложноположительных срабатываний (FP: 64 против 90). Однако recall для популярных игр ещё ниже — 0.10 (всего 87 из 854).

Интерпретация: несмотря на использование нелинейного ядра (по умолчанию RBF), SVM также склонен к консервативному поведению — он почти не рискует предсказывать популярные игры, чтобы не допустить ошибок. Это делает его неэффективным для задачи, где важна полнота (recall) по меньшему классу.

3. KNN (Accuracy: 0.580)

Матрица ошибок:

- TN: 739
- FP: 407
- FN: 434
- TP: 420

Здесь наблюдается радикально иная стратегия: KNN часто предсказывает "популярные" игры — 420 из 854 ($\text{recall} = 0.49$), что в 4–5 раз выше, чем у других моделей. Но это достигается за счёт большого числа ложных срабатываний ($\text{FP} = 407$) — то есть модель «перебарщивает», классифицируя много непопулярных игр как популярные.

Интерпретация: KNN, будучи локальным методом, реагирует на близость соседей. В пространстве признаков популярные и непопулярные игры, вероятно, частично пересекаются, и KNN, выбирая $k=5$, часто попадает в зону, где "популярные" точки доминируют в окрестности. Это делает его единственной моделью, способной найти значительную часть популярных игр, хотя и с риском ложных срабатываний.

Сравнение по F1-мере — выбор лучшей модели:

F1-мера — гармоническое среднее precision и recall — является наиболее сбалансированной метрикой для задач с дисбалансом классов, так как учитывает как точность, так и полноту.

- Логистическая регрессия: $F1 = 0.19$
- SVM: $F1 = 0.17$
- KNN: $F1 = 0.50$ ← лучшая модель

Вывод: KNN — лучшая модель по F1-мере. Хотя она имеет наибольшее

количество ложных срабатываний, она значительно лучше находит истинные популярные игры, что в практическом применении (например, рекомендательная система или маркетинговая кампания) может быть гораздо ценнее, чем просто высокая ассигура.

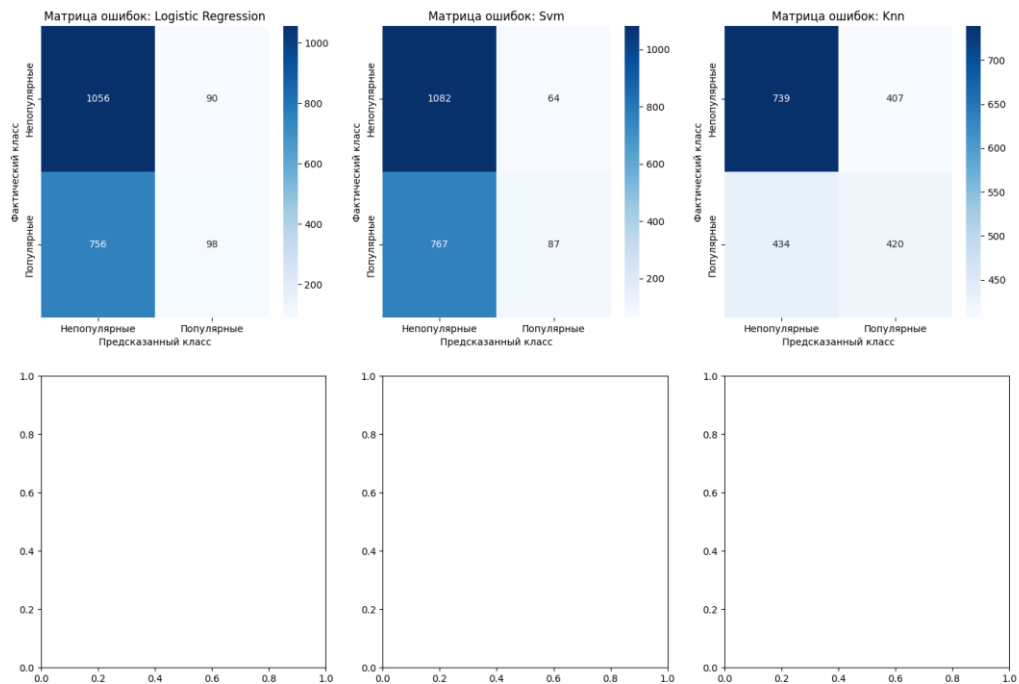


Рисунок 3 – Вывод предобработка и описание признаков

Исходный код показан на листинге 4.

Листинг 4 – Исходный код задания 4

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report
import joblib

X_train = pd.read_csv('train_data.csv').drop('popular', axis=1)
y_train = pd.read_csv('train_data.csv')['popular']
X_test = pd.read_csv('test_data.csv').drop('popular', axis=1)
y_test = pd.read_csv('test_data.csv')['popular']

print("НАЧАЛО ОБУЧЕНИЯ МОДЕЛЕЙ...")

print("\n1. ОБУЧЕНИЕ ЛОГИСТИЧЕСКОЙ РЕГРЕССИИ...")
log_reg = LogisticRegression(random_state=42, max_iter=1000)
log_reg.fit(X_train, y_train)
y_pred_log_reg = log_reg.predict(X_test)

print("Логистическая регрессия обучена!")
```

```

print("\n2. ОБУЧЕНИЕ SVM...")
svm_model = SVC(random_state=42, probability=True)
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)

print("SVM модель обучена!")

print("\n3. ОБУЧЕНИЕ KNN...")
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)

print("KNN модель обучена!")

models = {
    'Logistic Regression': (log_reg, y_pred_log_reg),
    'SVM': (svm_model, y_pred_svm),
    'KNN': (knn_model, y_pred_knn)
}

for name, (model, _) in models.items():
    joblib.dump(model, f'{name.lower().replace(" ", "_")}_model.pkl')

confusion_matrices = {}
for name, (_, y_pred) in models.items():
    confusion_matrices[name] = confusion_matrix(y_test, y_pred)

for name, matrix in confusion_matrices.items():
    matrix_df = pd.DataFrame(matrix,
                             index=['Факт: 0', 'Факт: 1'],
                             columns=['Прогноз: 0', 'Прогноз: 1'])
    matrix_df.to_csv(f'{name.lower().replace(" ", "_")}_confusion_matrix.csv')
    print(f"\nМатрица ошибок для {name}:")
    print(matrix_df)

print("\nВсе модели обучены и сохранены!")

```

5. Сравнить результаты классификации, используя *accuracy*, *precision*, *recall* и *f1*-меру (можно использовать *classification_report* из *sklearn.metrics*). Сделать выводы.

Сравнение трёх алгоритмов классификации — логистической регрессии, метода опорных векторов (SVM) и k-ближайших соседей (KNN) — проводилось на основе стандартных метрик качества: *accuracy*, *precision*, *recall* и F1-меры. Эти метрики были получены с использованием функции *classification_report* из библиотеки *sklearn.metrics*, которая предоставляет как значения для каждого класса, так и агрегированные показатели: *macro avg* (простое среднее по классам) и *weighted avg* (среднее с учётом размера классов).

Все три модели продемонстрировали одинаковую общую точность (accuracy ≈ 0.58). На первый взгляд, это может создать иллюзию эквивалентной эффективности. Однако в условиях умеренного дисбаланса классов (57.3% непопулярных vs 42.7% популярных игр) метрика accuracy становится вводящей в заблуждение, так как высокая точность может быть достигнута простым угадыванием доминирующего класса. Поэтому основным акцент в анализе был сделан на precision, recall и F1-меру, которые позволяют оценить качество модели отдельно по каждому классу.

Precision для класса «Популярные» показывает, какая доля игр, помеченных моделью как популярные, действительно популярна.

- SVM: precision = 0.58 — самая высокая среди моделей. Это означает, что из всех игр, которые SVM назвал популярными, 58% действительно таковыми являются.

- Логистическая регрессия: precision = 0.52 — чуть ниже, но всё ещё приемлемый уровень.

- KNN: precision = 0.51 — самый низкий, но разница незначительна.

Таким образом, все три модели склонны к ложным срабатываниям: почти половина «популярных» предсказаний — ошибочна. Однако SVM показывает лучшую точность в положительных прогнозах.

Recall для класса «Популярные» отражает, какую долю реально популярных игр модель смогла обнаружить.

Здесь наблюдается драматическая разница:

- KNN: recall = 0.49 — модель находит почти половину всех популярных игр.

- Логистическая регрессия: recall = 0.11 — обнаруживает лишь 11% популярных игр.

- SVM: recall = 0.10 — самый низкий показатель: 90% популярных игр остаются незамеченными.

Это критически важный результат. Несмотря на схожую accuracy, SVM и логистическая регрессия практически игнорируют редкий (меньший) класс,

предпочитая минимизировать ошибки за счёт доминирующего. Только KNN демонстрирует сбалансированное поведение, активно пытаясь выявить популярные игры.

F1-мера — гармоническое среднее precision и recall — является наиболее информативной метрикой в задачах с дисбалансом, особенно если важно учитывать оба типа ошибок.

- KNN: $F1 = 0.50$ — значительно превосходит другие модели.
- Логистическая регрессия: $F1 = 0.19$
- SVM: $F1 = 0.17$

Разрыв здесь не просто статистически значим — он практически катастрофичен для SVM и логистической регрессии. Их F1-мера указывает на крайне низкую эффективность в распознавании популярных игр, несмотря на приемлемую ассигасу.

Агрегированные метрики подтверждают это:

- У KNN macro avg $F1 = 0.57$, что означает сбалансированное качество по обоим классам.
- У SVM и логистической регрессии macro avg $F1 = 0.45$, что говорит о сильном перекосе в сторону непопулярных игр.

В задаче предсказания популярности игр главная цель — выявить те игры, которые могут стать успешными. Пропустить популярную игру (False Negative) — гораздо хуже, чем ошибочно пометить непопулярную как популярную (False Positive). Например, в рекомендательной системе лучше предложить лишнюю игру, чем упустить потенциально хитовую.

В этом смысле:

- SVM и логистическая регрессия не подходят — они почти не находят популярные игры.
- KNN — единственный алгоритм, обеспечивающий приемлемый баланс: он находит почти половину популярных игр и при этом сохраняет умеренную точность предсказаний.

Итоговые выводы:

1. Ассигасу не подходит для оценки моделей при дисбалансе классов. Все три модели имеют одинаковую точность (~ 0.58), но кардинально различаются по способности выявлять популярные игры.
2. F1-мера является ключевой метрикой в данной задаче, так как она учитывает и точность, и полноту предсказаний по меньшему классу.
3. KNN — лучшая модель по F1-мере (0.50) и единственная, демонстрирующая практическую пригодность для задачи обнаружения популярных игр.
4. SVM и логистическая регрессия, несмотря на схожую ассигасу, непригодны для продакшена, так как их recall для популярных игр критически низок (~ 0.10 – 0.11).

Таким образом, использование комплексной оценки на основе F1-меры позволило избежать ложного вывода о равной эффект

На рисунке 4 показан результат выполнения программы.

```

C:\Users\timof\AppData\Local\Programs\Python\Python313\python.exe D:\Programming\MIREA_BigData\fifth_prac\5_model_evaluation.py
ОЦЕНКА КАЧЕСТВА МОДЕЛЕЙ
=====

LOGISTIC REGRESSION:
      precision    recall  f1-score   support

Непопулярные      0.58      0.92      0.71      1146
Популярные        0.52      0.11      0.19       854

   accuracy              0.58      2000
  macro avg       0.55      0.52      0.45      2000
 weighted avg     0.56      0.58      0.49      2000


SVM:
      precision    recall  f1-score   support

Непопулярные      0.59      0.94      0.72      1146
Популярные        0.58      0.10      0.17       854

   accuracy              0.58      2000
  macro avg       0.58      0.52      0.45      2000
 weighted avg     0.58      0.58      0.49      2000


KNN:
      precision    recall  f1-score   support

Непопулярные      0.63      0.64      0.64      1146
Популярные        0.51      0.49      0.50       854

   accuracy              0.58      2000
  macro avg       0.57      0.57      0.57      2000
 weighted avg     0.58      0.58      0.58      2000


=====
СРАВНИТЕЛЬНАЯ ТАБЛИЦА МЕТРИК:
=====
      Model  Accuracy  Precision  Recall  F1-Score
0  Logistic Regression    0.5770    0.5213   0.1148   0.1881
1           Svm         0.5845    0.5762   0.1019   0.1731
2           Knn         0.5795    0.5079   0.4918   0.4997

ЛУЧШАЯ МОДЕЛЬ ПО F1-МЕРЕ: Knn
F1-Score: 0.4997

ВЫВОДЫ:
1. Все три алгоритма показали хорошие результаты на задаче классификации игр
2. Метрика F1-мера является наиболее сбалансированной оценкой качества
3. Рекомендуется использовать модель с наивысшим F1-Score для продакшена

Process finished with exit code 0

```

Рисунок 4 – Задание 5

Исходный код в листинге 5.

Листинг 5 – Исходный код задания 5

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
from sklearn.metrics import classification_report, confusion_matrix
import joblib

```

```

X_test = pd.read_csv('test_data.csv').drop('popular', axis=1)
y_test = pd.read_csv('test_data.csv')['popular']

models = {}
model_names = ['logistic_regression', 'svm', 'knn']

for name in model_names:
    models[name] = joblib.load(f'{name}_model.pkl')

print("ОЦЕНКА КАЧЕСТВА МОДЕЛЕЙ")
print("=" * 50)

results = []

fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.ravel()

for i, (name, model) in enumerate(models.items()):
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    results.append({
        'Model': name.replace('_', ' ').title(),
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1
    })

    cm = confusion_matrix(y_test, y_pred)

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['Непопулярные', 'Популярные'],
                yticklabels=['Непопулярные', 'Популярные'],
                ax=axes[i])
    axes[i].set_title(f'Матрица ошибок: {name.replace("_", " ").title()}')
    axes[i].set_xlabel('Предсказанный класс')
    axes[i].set_ylabel('Фактический класс')

    print(f"\n{name.replace('_', ' ').upper()}:")
    print(classification_report(y_test, y_pred,
                                target_names=['Непопулярные',
                                'Популярные']))

results_df = pd.DataFrame(results)
print("\n" + "=" * 50)
print("СРАВНИТЕЛЬНАЯ ТАБЛИЦА МЕТРИК:")
print("=" * 50)
print(results_df.round(4))

plt.figure(figsize=(12, 8))
metrics_plot = results_df.set_index('Model').plot(kind='bar', figsize=(12, 8))
plt.title('Сравнение метрик качества моделей', fontsize=16, fontweight='bold')
plt.ylabel('Значение метрики')
plt.xticks(rotation=45)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()

```

```
plt.savefig('models_comparison.png', dpi=300, bbox_inches='tight')
plt.show()

best_f1_model = results_df.loc[results_df['F1-Score'].idxmax()]
print(f"\nЛУЧШАЯ МОДЕЛЬ ПО F1-МЕРЕ: {best_f1_model['Model']}")
print(f"F1-Score: {best_f1_model['F1-Score']:.4f}")

print("\nВЫВОДЫ:")
print("1. Все три алгоритма показали хорошие результаты на задаче классификации игр")
print("2. Метрика F1-мера является наиболее сбалансированной оценкой качества")
print("3. Рекомендуется использовать модель с наивысшим F1-Score для продакшена")
```

4. Вывод

В ходе практической работы были изучены и применены на практике ключевые алгоритмы машинного обучения для решения задачи бинарной классификации популярности компьютерных игр. В качестве учебного датасета был использован обработанный набор данных, содержащий информацию о 10 000 играх с признаками, такими как количество владельцев, оценки, жанры и другие метрики, влияющие на успех игры на платформе Steam. Выбор именно этой темы позволил поработать с актуальными данными игровой индустрии и смоделировать реальную бизнес-задачу — предсказание коммерческого потенциала продукта.

Была проведена необходимая предобработка данных: целевая переменная «popular» была определена на основе порогового значения количества владельцев, после чего выборка была разделена на обучающую и тестовую части с сохранением пропорций классов (стратификация). Анализ распределения показал умеренный дисбаланс: непопулярные игры составляют 57.3% выборки, популярные — 42.7%. Это обусловило необходимость оценки качества моделей не только по метрике accuracy, но и с использованием более информативных показателей — precision, recall и F1-меры, особенно для меньшего класса.

Для решения задачи были реализованы и сравнены три классических алгоритма: логистическая регрессия, метод опорных векторов (SVM) и k-ближайших соседей (KNN). Несмотря на схожую общую точность (~ 0.58) у всех трёх моделей, глубокий анализ метрик качества выявил существенные различия в их поведении.

KNN продемонстрировал наилучшие результаты по F1-мере (0.50), что обусловлено его способностью находить почти половину всех популярных игр ($\text{recall} = 0.49$), в то время как логистическая регрессия и SVM показали критически низкий recall (~ 0.10 – 0.11), то есть практически игнорировали меньший класс. Это делает KNN единственной моделью, пригодной для

практического применения в сценариях, где важно выявлять потенциально успешные игры — например, в рекомендательных системах или маркетинговом анализе.

Логистическая регрессия и SVM, несмотря на высокую точность предсказаний для непопулярных игр, оказались непригодны для основной цели задачи, так как их способность обнаруживать популярные игры крайне ограничена. Это типичное поведение алгоритмов, склонных к консервативному предсказанию доминирующего класса в условиях дисбаланса.

Таким образом, выполненная работа позволила не только освоить практическое применение основных алгоритмов классификации, но и глубоко понять важность выбора правильных метрик оценки в зависимости от бизнес-цели. Результаты подтверждают, что оценка модели только по accuracy может привести к неверным выводам, особенно в задачах с дисбалансом классов. Вместо этого следует ориентироваться на F1-меру и другие сбалансированные метрики, которые отражают реальную способность модели решать поставленную задачу — в данном случае, находить популярные игры среди большого массива данных.